CourtBook

Demo and viva notes

Cloud Computing project · Azure App Service · 5-minute talk and viva answers

The project in one line

CourtBook is a live sports court booking app for our campus, hosted on Azure App Service. It uses the PaaS model, stops double booking under real concurrent use, and measures its own service levels.

Live site: courtbook-brooxxhh-g2hug8hkhnaze3gq.indiasouthcentral-01.azurewebsites.net

The 5 minutes at a glance

Time	Topic	Show on screen
0:00	What it is	Book page on the live site
0:30	Proof of Azure	Azure Portal Overview, then the About page
1:30	Live booking and double booking	Classmates' phones, then two phones together
2:30	Admission control	Third booking gets rejected
3:15	MapReduce	Admin page
4:00	SLA and monitoring	SLA page
4:40	Close	Back to the Azure Portal

Before ma'am arrives

- Open the live site 5 minutes early. The Free plan sleeps when idle, and the first load is slow.
- Log in to **portal.azure.com** and leave the app's Overview page open in a tab.
- Open the live site's **Admin**, **SLA** and **About** pages in separate tabs.
- Put the site address on a QR code so classmates can scan it.
- Make one or two test bookings so the Admin tables aren't empty.
- Keep screenshots of every page ready in case the Wi-Fi fails.

The three things to remember

Idea	One-line version
PaaS	We give Azure the code, and Azure runs the servers, the OS and Python.
Atomic slot lock	The database refuses a second booking for the same slot, so only one person can win.
Measured reliability	The app measures its own response times and availability against a 99.9% target.

The 5-minute talk

Say this in your own words. It's written to fit 5 minutes at a calm pace, with time for tapping and switching tabs.

0:00 – 0:30 · What it is

Show: Book page

Ma'am, our project is CourtBook, a court booking system for the campus. Students pick a court and a 30-minute slot from their phones, and everyone sees the same live grid. It's a real deployed cloud application, not one running on my laptop.

0:30 – 1:30 · Proof of Azure

Show: Portal Overview, then About page

This is our app in the Azure Portal. It's Azure App Service, running on Linux on the Free F1 plan. This is Platform as a Service: we uploaded the code, and Azure manages the servers, operating system and Python. Now here's our About page. This card is read live from Azure by the running app, so you can see the app name, region and plan. The table below shows the matching AWS and Google Cloud services.

1:30 – 2:30 · Live booking

Show: Classmates scan the QR code, then two phones book one slot

Please scan this and book a slot. You can see the grid update by itself, because every phone refreshes every 3 seconds. Now the main problem in any booking system: two people booking the same slot at the same moment. We'll tap together. Only one of us gets it, and the other sees "Slot just taken by someone else". The database has a unique rule on court, date and time, so the second insert fails. That makes it an atomic lock.

2:30 – 3:15 · Admission control

Show: Third booking on one registration number

Next is admission control, which decides who gets in before the work is done. Each student can book 2 slots a day, so this third one is denied. Past slots are refused too. And if the server is overloaded, extra requests get a 503 "Server busy" reply, so it stays healthy instead of crashing.

3:15 – 4:00 · MapReduce

Show: Admin page

The Admin page shows utilisation for each court. This report finds the peak hours and the most popular court using MapReduce. Map emits a key and a 1 for each booking's court and hour. Shuffle groups the same keys together. Reduce adds them up. All three tables are shown so you can see the data flow.

4:00 – 4:40 · SLA and monitoring

Show: SLA page

This page tracks service levels. Every request is timed, so we show the average, the 95th percentile, and the share under our 500 millisecond target. A background thread checks our health endpoint every 30 seconds to calculate availability. For a 99.9% target, we're allowed 43.2 minutes of downtime a month. The badge shows whether we're meeting it.

4:40 – 5:00 · Close

Show: Azure Portal

To sum up, CourtBook shows PaaS deployment, persistent storage, concurrency control, admission control, MapReduce and SLA monitoring in one working app. Thank you, ma'am. We're ready for your questions.

If she gives you one more minute

Restart the app. Click Restart on the portal's Overview page and wait about a minute. The bookings are still there because they're on Azure's persistent storage, but the SLA counters reset because they were in memory.

Show the Log stream. Open Monitoring, then Log stream, and book a slot. The request appears in Azure's logs.

If something fails

Problem	What to do
Site is slow to open	Say: "The Free plan sleeps when idle, so this is a cold start." Wait up to a minute.
Azure 503 page	Azure is starting the app. Wait and reload. Use the portal in the meantime.
Wi-Fi fails	Use mobile data, then your screenshots.
Badge shows "SLA violated"	Explain it calmly: one failed health check is enough to drop below 99.9% when few checks have run. Honest measurement is the point of the page.

Viva questions and answers

Keep answers short, and speak from what you built. If you don't know something, say so, then explain what you do know.

Cloud basics

What is cloud computing?
Using computing resources such as servers, storage and databases over the internet, on demand, and paying for what you use, instead of owning the hardware.

What are IaaS, PaaS and SaaS?
IaaS gives you virtual machines and you manage the OS (Azure Virtual Machines). PaaS runs your code and manages the platform (Azure App Service). SaaS is a finished application you just use (Gmail).

Which one is your project, and why?
PaaS. We only provide code and settings. Azure handles the servers, OS patches and Python runtime. To the students who use it, CourtBook itself behaves like SaaS.

Which deployment model did you use?
Public cloud. It runs on Microsoft's shared infrastructure.

What is the difference between scalability and elasticity?
Scalability is the ability to handle more load by adding resources. Elasticity means adding and removing them automatically as demand changes.

What is the difference between scaling up and scaling out?
Scaling up gives one instance more power. Scaling out adds more instances behind a load balancer.

Azure and deployment

What is a resource group?
A folder that holds related Azure resources, so they can be managed or deleted together.

What is an App Service plan?
It defines the region, compute and pricing that the app runs on. Ours is Linux, Free F1.

How did you deploy the app?
From VS Code with the Azure App Service extension. It uploads the code, Azure installs requirements.txt, and then it runs our startup command.

Explain your startup command.
gunicorn is the production web server. **--bind=0.0.0.0** listens on all network interfaces. **--timeout 600** allows slow requests. **--workers 1 --threads 8** means one process with 8 threads. **app:app** is the app object in app.py.

Why Gunicorn, not Flask's own server?
Flask's built-in server is meant for development only. Gunicorn is built for production traffic.

Why one worker and 8 threads?
Our counters are in memory, and each worker is a separate process with separate memory. Threads share memory, so all requests update the same numbers.

What are the limits of the Free F1 plan?
About 60 CPU minutes a day, no Always On so it sleeps when idle, no scaling out, and no Azure SLA.

What is a cold start?
The delay on the first request after an idle app has been put to sleep, because Azure has to start it again.

Did you face any problems during deployment?
Yes. Creating the app failed because the subscription wasn't registered for Microsoft.OperationalInsights, so we registered that resource provider. We also saw a temporary 503 from Azure while the app was starting, and we learned that Azure adds a random suffix to new app addresses.

Storage and concurrency

Where is the data stored?
In an SQLite file at /home/data/courtbook.db. /home is Azure's persistent storage, which survives restarts.

What is a race condition?
When the result depends on the timing of parallel operations. Two users could both see a slot as free and both try to book it.

How do you prevent double booking?
With UNIQUE(court_id, date, slot_time). The database accepts the first insert and rejects the second with an error. We show that as "Slot just taken".

Why not check in Python first, then insert?
Two requests could both pass the check before either one inserts. The constraint makes the check and the write one atomic step.

What does BEGIN IMMEDIATE do?
It takes SQLite's write lock at the start of the transaction. Our daily-limit count and the insert then happen together, so parallel requests can't slip past the limit.

What is ACID?
Atomicity (all or nothing), Consistency (rules always hold), Isolation (transactions don't interfere) and Durability (committed data survives).

How did you test this?
We sent 30 parallel bookings for one slot. Exactly one succeeded and the rest were rejected. We ran the same test with the real Gunicorn command.

Admission control

What is admission control?
Deciding whether to accept a request before doing the work, to protect the system and keep things fair.

What are your three checks?
A limit of 2 bookings per student per day, no past slots, and a concurrency limit that returns HTTP 503.

Which HTTP codes do you use?
201 for a booking, 400 for a past slot or bad input, 403 for the daily limit, 409 for a slot conflict, and 503 for server busy.

Why reject requests instead of queueing them all?
Queues grow under overload, and every user gets slow. Rejecting early keeps the service fast for those admitted. This is called load shedding.

The brief says 20. Why is yours set to 6?
The startup command gives 8 threads, so only 8 requests are inside the app at a time, and the rest queue in Gunicorn. The counter could never pass 20. We confirmed this with a load test, then set MAX_CONCURRENT to 6 so that the rejection can really happen. The code default is still 20.

MapReduce

What is MapReduce?
A model for processing large data in parallel. Map emits key-value pairs, Shuffle groups them by key, and Reduce combines each group.

Give an example from your project.
A 6 PM Gym booking emits (Court: Gym, 1) and (Hour: 6 PM, 1). Shuffle gives Court: Gym with [1, 1, 1]. Reduce gives Court: Gym = 3.

Is your version really distributed?
No. It runs on one machine to show the flow clearly. Hadoop or Spark would run the same phases across many machines, which matters at much larger data sizes.

SLA and monitoring

What is the difference between SLA, SLO and SLI?
An SLI is the measurement, such as response time. An SLO is the target, such as under 500 ms. An SLA is the formal promise, such as 99.9% availability.

What is the 95th percentile, and why use it?
95% of requests were faster than this value. Averages hide slow requests, but p95 shows what the slower users experience.

How is availability calculated?
Passed health checks divided by total health checks, times 100. We run one check every 30 seconds.

How do you get 43.2 minutes?
99.9% allows 0.1% downtime. A 30-day month has 43,200 minutes, and 0.1% of that is 43.2 minutes.

When does the badge turn red?
When availability falls below 99.9%, or p95 goes above 500 ms.

Does Azure give you a 99.9% SLA?
No. Azure doesn't offer an SLA on the Free plan. 99.9% is our own target for the project.

What are the weaknesses of your health check?
It runs inside the app, so if the whole app is down, nothing records the failure. The numbers also reset on restart. A real system would use an outside monitor such as Azure Application Insights.

What is the difference between /sla and Azure Monitor?
/sla is measured inside our app. Azure Monitor is measured by the platform, from outside our code.

Design choices and improvements

Why SQLite and not a cloud database?
It's free and needs no separate server, which fits a Free plan demo. In production we would use a managed database such as Azure SQL or Azure Database for PostgreSQL.

Why polling every 3 seconds, not WebSockets?
Polling is simple and works reliably on the Free plan. The downside is the extra traffic: 60 phones make about 20 requests a second. Hidden tabs stop polling to reduce that.

What are the AWS and GCP equivalents?
App Service: Elastic Beanstalk and App Engine. Persistent database: RDS and Cloud SQL. Azure Monitor: CloudWatch and Cloud Monitoring.

How would you scale this for the whole university?
Move to a paid plan, replace SQLite with a managed database, keep counters in a shared store such as Azure Cache for Redis, then add instances behind the load balancer.

What security weaknesses does it have?
/admin has no login, and anyone who knows a registration number can view and cancel those bookings. A real version would use college login, such as Microsoft Entra ID. The app also limits exposure: the public grid shows only first names, and page text is safely escaped.

What would you add next?
College login, a managed database, Application Insights alerts, and email reminders for bookings.